

MPM の粒を面に戻す — 27% 太る。膨らむ厚みは粒の間隔の 0.4倍

2026-09-14 | Houdini 21.0.700 | /obj/mpm | ノード 5 / 接続 4

何を作ったか

実験061・062で、MPM は物を粒の集まりとして持つと分かった。ただし粒のままではレンダできない。mpmsurface で面に戻す。

ここで確かめられることがある。体積 1.0 の箱を粒にして、面に戻したら、また 1.0 になるか。実験059で obj の往復を測ったのと同じ考え方で、今度は形式ではなく「粒を経由すること」で何が変わるかを見る。

結果は 1.2663。27% 太る。理由は絵を見れば分かる。面は粒の一つひとつを外側から包むので、元の箱より一回り大きくなる。

その厚みも出せる。1辺が $1 + 2d$ に太ったと考えて逆算すると、粒の間隔 0.16 で $d = 0.05774$ 、0.08 で $d = 0.03538$ 。間隔で割ると $0.3609 / 0.4261 / 0.4094 / 0.4423$ 。つまり膨らむ厚みは粒の間隔の 0.4倍くらいで、間隔を細かくすれば太り方も減る。

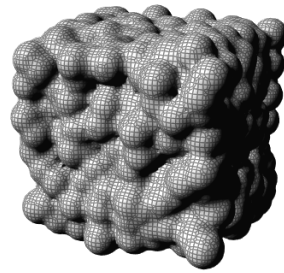
面の細かさ (voxelscale) はほとんど効かない。1.0 / 0.75 / 0.5 で体積は 1.2663 / 1.2705 / 1.2736 と 0.6% しか変わらないのに、点の数は 5,495 から 23,485 へ 4.3倍になる。しかも 1.5 と 2.0 では何も出てこない。

もうひとつ。面の作り方には「VDB from Particles」のほかに「Neural Point Surface」という機械学習の方法がある。こちらは点が 2,168 と 2.5分の1なのに体積は 1.1882 で元に近い。ただし 1.35秒かかり、3倍近く遅い。

ノード	タイプ	デフォルトから変更したパラメータ
box	box	(変更なし)
container	mpmcontainer	particlesep = 0.1 sizex = 4.0 sizey = 4.0 sizez = 4.0
source	mpmsource	(変更なし)
surface	mpmsurface	outputtype = 2 voxelscale = 0.75
to_polygons	convert	(変更なし)

粒を包むから太る

1辺 1.0 の箱を、粒の間隔 0.1 で 1,001個の粒にしてから面に戻した。



Houdini

Houdini

voxelscale 2.0。粗すぎて何も出てこない（点 0 / 面 0）。

voxelscale 0.5。23,485点。粒ひとつずつの丸みがそのまま面になる。

voxelscale	点	面	体積	元との比	秒
2.0	0	0	0.000000	—	0.21
1.5	0	0	0.000000	—	0.21
1.0	5,495	5,488	1.266304	1.2663	0.22
0.75	10,180	10,176	1.270461	1.2705	0.46
0.5	23,485	23,493	1.273598	1.2736	0.45

粒の間隔を変えると、太り方はどう変わるか

voxelscale は既定のまま。1辺が $1 + 2d$ に太ったと考えて d を逆算した。

粒の間隔	粒	点	面	体積	元との比	$1 + 2d$	d	$d \div \text{間隔}$
0.16	231	2,278	2,272	1.387964	1.3880	1.11547	0.05774	0.3609
0.12	584	3,957	3,950	1.339217	1.3392	1.10226	0.05113	0.4261
0.10	1,001	5,495	5,488	1.266304	1.2663	1.08188	0.04094	0.4094
0.08	1,956	8,589	8,574	1.227668	1.2277	1.07076	0.03538	0.4423

面の作り方を考える

同じ粒（1,001個）から、2つの方法で面を作った。

やり方	点	面	体積	元との比	秒
VDB from Particles	5,495	5,488	1.266304	1.2663	0.47
Neural Point Surface	2,168	2,356	1.188212	1.1882	1.35

わかったこと

- ・粒を経由すると 27% 太る。体積 1.0 の箱が 1.266304 になる。面は粒の一つひとつを外側から包むので、元より一回り大きくなる。

- ・膨らむ厚みは、粒の間隔の 0.4 倍くらいだった。1 辺が $1 + 2d$ に太ったと考えて逆算すると、 $d \div$ 間隔が $0.3609 / 0.4261 / 0.4094 / 0.4423$ 。4 通りで $0.36 \sim 0.44$ に収まる。

- ・だから細かくすれば太り方は減る。間隔 0.16 で 1.3880、0.08 で 1.2277。間隔を半分にすると、太る割合もおおよそ半分になる。

- ・面の細かさ (voxelscale) はほとんど効かない。1.0 / 0.75 / 0.5 で体積が 1.2663 / 1.2705 / 1.2736。0.6% しか変わらない。

- ・それなのに点の数は 4.3 倍になる。5,495 $\rightarrow$ 23,485。重くなるだけで、形はほとんど変わらない。太り方を減らしたいなら、voxelscale ではなく粒の間隔を触る。

- ・粗すぎると何も出てこない。voxelscale 1.5 と 2.0 では点も面も 0。エラーは出ない。「0 個」も結果として記録しておく。

- ・mpmsurface が出すのは polysoup。1 プリミティブの中に全部の面が入っている。そのままだと面ごとの計算ができないので、convert でふつうのポリゴンに割ってから測った。最初これに気づかず、体積が 0.06 という妙な値になった。

- ・機械学習の方法もある。「Neural Point Surface」。点が 2,168 と 2.5 分の 1 なのに、体積は 1.1882 で元に近い。軽くて正確。

- ・ただし 3 倍近く遅い。0.47 秒 対 1.35 秒。1 フレームならどうでもいい差だが、200 フレームなら 3 分 対 4.5 分になる。

- ・出力の既定は面ではない。outputtype の既定は「Surface VDB」で、面が欲しいなら polygonmesh に変える。これも最初に掛かった。

次にやること

- ・Neural Point Surface の精度を、もっと複雑な形で測る
- ・MPM の時間の刻み (substep) と結果の関係
- ・粒の間隔と、シミュレーションそのものの結果の関係